Care Note — Round Two

Response to the sixteen clinic scenarios and the twelve-capability list. Architecture, schema and latency are unchanged and remain in `TECHNICAL_BRIEF.md`; this covers what the review changed. Per-scenario verdicts and their tests are in `SCENARIO_COVERAGE.md`.

Where we landed: 9 SURVIVES · 6 PARTIAL · 1 DOES NOT on the scenarios, from 6 · 6 · 4 in our own first assessment. Decisions D-070 to D-103, 542 backend test functions (866 parametrised cases) and 67 component tests passing.

Section 11 is a final pass, and it found four more defects. All four were live while the suite was green. Two of them were misfiring on the two surfaces a patient can actually see. We have left the account of them at full length rather than folding them into a summary, because how they were found matters more than that they were fixed.

Two rows moved backwards, deliberately. Scenario 3 was SURVIVES and is now PARTIAL: a patient's phone number was travelling in a query string and therefore into the access log. Scenario 2 was SURVIVES and is now PARTIAL: we had answered *where* clinic isolation is enforced and never measured what happens when that one line is wrong — the answer is every patient in both clinics, and nothing else catches it. Sections 7 and 8 cover both.

`pytest tests/test_survival_scenarios.py -v` walks the sixteen scenarios one at a time, and fails if its verdicts drift from the table below.

1. What the review actually found

Our own assessment came out 6/6/4 before any code changed. The pattern in those failures was more useful than any individual fix: **what survived was structural, what failed was operational.** Access control fused into a type, redaction fused into the only code that can reach a network, highlights anchored to a version — all held. What failed was every question of the form *what happens when*: when the provider is down, when the server crashes, when a clinic needs to add a patient on a Tuesday. We had built a record system and had not built the clinic around it.

Two defaults explain most of it, and both are worth stating because they generalise beyond this build.

The stub LLM provider cannot fail. It is in-process, so it cannot time out, refuse a connection or return a 503. Every test run and demo for the entire build executed against a provider physically incapable of failing — which is why a provider outage turned out to be an unhandled 500 with a traceback. The default is still right; it makes the build runnable without an API key. But it bought that determinism by removing the failure surface from view, and nothing made the trade visible. `CARENOTE_LLM_FORCE_UNAVAILABLE` now exercises that path on every run.

The seed script stood in for features. `init_db.py` runs at Phase 1 step 1, so from the first commit every test and demo began with a populated database. *How does a patient come to exist?* never arose, because patients always already did. Anything a seed provides is a feature you have not built and will not notice missing — which is exactly scenario 1.

2. Scenario verdicts

#	Scenario	Was	Now
1	Patient with no email	PARTIAL	**SURVIVES** — staff enrolment, phone as identifier
2	Clinic isolation, one line	SURVIVES	**SURVIVES** — `AccessScope.query()`, three test files
3	Read your logs	DOES NOT	**PARTIAL** — crash path fixed; a URL leak found later (§7)
4	Prove the ordering	SURVIVES	**SURVIVES** — one exit, asserted by source scan
5	Clinic B on Monday	PARTIAL	**PARTIAL** — zero schema changes; config surface built, vocabulary still global
6	Trilingual consult	PARTIAL	**PARTIAL** — parity for en/ms; Hokkien flagged, not read
7	Allergy at minute two	DOES NOT	**DOES NOT** — batch; boundary asserted by test
8	Model hangs 45s	PARTIAL	**PARTIAL** — 8s timeout and cancel; no server-side abort
9	Provider 503 for an hour	DOES NOT	**SURVIVES** — degrades to extractive, and the card says so
10	Two people, same note	SURVIVES	**SURVIVES** — version check plus unique constraint
11	Link never received	DOES NOT	**PARTIAL** — reach modelled; no sender exists
12	Summary wrong by one dosage	PARTIAL	**SURVIVES** — correction banner and a dose gate
13	Allergy asserted vs denied	PARTIAL	**SURVIVES** — `assertion_vs_denial` at HIGH, one card per disagreement
14	A number that means nothing	SURVIVES	**SURVIVES** — floor now language-independent
15	Ranking learns from what it showed	SURVIVES	**PARTIAL** — floors, exploration and inspectability hold; bias now *measured* at 0.77 rather than argued (D-092)
16	Highlight cites edited source	SURVIVES	**SURVIVES** — now side by side, both versions named

On the twelve capabilities: **4 SURVIVES · 7 PARTIAL · 1 DOES NOT**, revised down from 6 · 5 · 1 by a second audit (§10). The single DOES NOT is streaming ASR. Full per-row detail with the tests that cover each is in `SCENARIO_COVERAGE.md`.

Four rows moved down, and the reasons differ. Two were overclaims: the correction leg of fact extraction was justified by "version history as the correction record", which is a human editing a note and not a speaker correcting themselves in a transcript; and exposure bias was marked SURVIVES on the strength of a mitigation existing, when the capability asks for an evaluation. One hid a defect (§10). The fourth, audience-appropriate output, is a capability we **declined on purpose** — no machine-written text can become patient-facing at all (D-067) — and ticking it would have claimed a feature where we had made an argument. Stated as a refusal it is the stronger answer; stated as a tick it is the thing the feedback said counts against us.

3. What changed

Failure handling (3, 8, 9). The chokepoint now translates timeouts, transport errors, 5xx and 429 into one `LLMUnavailableError`; it does not decide what to do about them. Each caller chooses, because the right answer differs by purpose: the scribe degrades to the deterministic extractive summariser it already had — labelled `offline-extractive-v1:provider-unavailable`, so a degraded note is legible as degraded — and a patient-facing generator would refuse. 4xx stays loud: a bad API key hiding behind a slightly worse summary is indistinguishable from an outage. Timeout 60s → 8s, plus a cancel button, since the transcript is stored before the model is called and abandoning loses the summary, never the consult.

Crash logging is a separate chokepoint: type, route and an eight-character reference, never `str(exc)`, which is where SQLAlchemy puts bound parameters.

Language parity (6, 14). The risk floor now works over canonical tags rather than English strings, so it inherits every language the tagger knows. It also gained negation handling, asymmetrically: a symptom is dropped only if it appears *exclusively* inside a negation, so "no chest pain Monday, chest pain today" still rates high, and a denied symptom drops to `medium` rather than to nothing. Content in an unsupported language is flagged as unread rather than silently producing nothing — abstention beats confident silence.

Denials as claims (13). Negated allergy mentions become `allergy_denial` claims instead of being discarded, and `assertion_vs_denial` fires at HIGH — not CRITICAL, because nothing dangerous has happened yet and diluting CRITICAL would break the level that means *someone is about to be given a drug they react to*.

Reach (11, 12). `unread` / `read` / `corrected`, derived from timestamps `PatientView` had recorded since D-033 and nothing had queried. `dispatched` is deliberately absent because nothing dispatches. The patient view leads with a plain-language correction banner, computed before the read marker moves.

Enrolment (1, 5). Staff can register a patient and issue a login; identifier type is explicit and a phone number is first-class. `Patient.dob` and `mrn` became nullable — both were `NOT NULL`, which was a second and quieter way the schema decided that someone known only by a phone number was not a patient.

Regeneration (capability list). Was undefined behaviour, found by probing: a new session id produced a duplicate summary, the same one crashed on a unique constraint. Now it reuses the entry and appends a version, so accepted highlights and comments survive — and refuses outright when a human has edited, because merging would mean choosing which of a clinician's sentences to keep.

Dosage (capability list, and the hint). Doses were compared against each other and never against a reference, so the build could see two entries disagree and not see 5000mg. A 17-drug adult reference now bands doses plausible / unusual / implausible; only implausible gates, and only on patient-facing writes, with the override recorded. Acknowledgement rather than refusal: a hard block teaches people to route around the check.

Provenance (16). Stale highlights now show the anchored text and the current text side by side, with both version numbers named.

4. What we tried that did not work

An order-of-magnitude dose threshold. Metformin 5000mg passed as merely `unusual` — the exact case the hint describes. The fix is principled rather than tuned: ranges are *single-dose*, a legitimate daily total reaches ~3× that (TDS), so beyond 3× exceeds any plausible daily total. 1500mg BD correctly stays unusual.

`@app.exception_handler(Exception)` for the log leak. Does not work. Starlette's `ServerErrorMiddleware` calls the handler and then *re-raises* so the server can log the traceback — sanitising the response and leaving the leak untouched. It had to be middleware. We only found this because the test asserted on captured log output rather than on the response body.

A blanket-denial pattern that matched too much. `denies allergy to aspirin` registered as a denial of *all* allergies. A contradiction detector that cries wolf is worse than one with gaps: the gap loses a finding, the false positive teaches people to stop reading all of them.

Streaming capture — not attempted. Retrofitting incremental extraction is tractable; `test_capture_timing.py` asserts the deterministic extractors work on two turns alone. What we could not resolve is the interruption policy. When it is acceptable to interrupt a doctor mid-consultation is a clinical workflow decision, and guessing would have produced a demo rather than a product.

5. Assumptions that no longer stand

"Logging is solved because `log_event` is content-free." Solved for content we log on purpose; silent about content logged on our behalf. One unhandled 500 put a name, an NRIC and note content in a single line. The earlier brief claimed "logs grepped, zero hits" — true of the success path, and the wrong question.

"Multilingual is fixed because the tagger handles Malay" (D-058). The fix landed at the tagger; the risk floor was a separate path making the same English-only assumption. Every Malay fixture passed because they only exercised the tagger. *When you fix an assumption at one layer, go looking for other layers that made the same one.*

"Dropping negated mentions is correct." Correct for its purpose, and it discarded the patient's denial entirely — a denial is a position, not an absence.

"The medication cue vocabulary is adequate." Built against written notes. *I will start you on amoxicillin* — how prescribing is actually said — classified as a bare dose claim and never reached the allergy comparison.

Still standing: regex redaction over NER for auditability (D-012); no HTML-escaping on write, because corrupting `dose <5mg` is worse than the XSS it prevents (D-015); full snapshots over diffs (D-006); least-privilege on staff visibility (D-004); and reporting contradictions without ever resolving them (D-068) — tested hardest, and the one we would defend most strongly.

6. Where we expect this to fail next

1. **A consult beyond English and Malay.** The unread flag means the system says so, but the content is still invisible downstream. First week.
2. **A dose the 17-drug table does not know.** No check at all, and the absence looks identical to a pass.
3. **Alert fatigue on contradictions.** Four classes now fire; nothing monitors the dismissal rate, which is the signature `NEVER_DAMPENED` exists to survive.
4. **Regeneration refusing too often.** Any human edit blocks it, including a typo fix — possibly annoying enough to teach people not to edit AI summaries.
5. **Two disagreeing clinicians in the learning loop.** Untested rather than designed; saturation bounds the damage.

Still not built, plainly: streaming ASR, acoustic diarization, any message sender, per-clinic configuration, presence indicators, and a real drug database. Scenario 7's *timing* remains DOES NOT — nothing is incremental, and `test_capture_timing.py` asserts that boundary rather than papering over it. Its *detection* half was a defect rather than a boundary, and is fixed (§10).

7. Re-auditing our own answers

Everything above was written, then the build was probed again as if by someone trying to disprove it. Three defects came out, **all inside rows already marked SURVIVES**, and none caught by the 509 tests passing at the time. The common cause is worth more than the fixes: **each test used the minimal shape of the case it was testing**, and the minimal shape is the one where the bug cannot appear.

Contradictions fanned out N×M (D-081). Every test used one assertion and one denial — the single shape where pairwise and grouped output are identical. A real chart re-records an allergy at every visit, so four "allergic to penicillin" entries against two denials produce eight findings that all say one thing. The card caps at five, so the copies filled the cap and an unrelated **metformin dose disagreement was evicted entirely** — a real conflict made invisible by a different one being mentioned more often, degrading as the record grew. Detection stays pairwise; the display unit is now `(kind, subject)` and every supporting entry keeps its own pointer, because "and 7 others" without links would trade an alert-fatigue problem for a provenance one.

Degradation was legible to an auditor, not a clinician (D-082). Section 3 claimed a degraded note is "legible as degraded." True of the database. In the interface the only trace was `ai_model_used` as a 10px grey monospace string in the provenance footer — a machine-facing identifier where a clinician looks least — so during an hour-long outage the card read as an ordinary AI summary. `ai_degraded` is now a chip, kept separate from confidence: "the model was unsure" and "no model read this consult" call for different responses.

A phone number in a query string (D-083). `POST /patients/{id}/login` took the patient's identifier as a query parameter, and for scenario 1 that identifier *is* a phone number. The ASGI access log records the full request line before the application sees it, and identically for requests that 404 or are refused by RBAC, so neither `log_event` nor the error middleware touches it. **The feature built to answer scenario 1 leaked through the door scenario 3 asks about.** That is why scenario 3 is now PARTIAL. `tests/test_url_surface.py` now asserts across every route that path and query parameters come from an explicit allowlist, with a second test asserting the allowlist stays free of PHI-shaped names so the invariant cannot be satisfied by widening it.

What this says about the rest of the table. Three overclaims in rows we had tested and believed, found in one pass. We would expect a fourth to exist. The rows we would probe next are **12 and 15** — the dosage gate has only been exercised against drugs the 17-entry table knows, and the learning floors have never met a clinic dismissing at a realistic rate over months.

8. The themes underneath the sixteen

Read as eight themes rather than sixteen incidents, the scenarios sort the build into two piles, and the split is not where we expected.

Enforcement that is strong but singular (theme 2). Access control is fused into a type and redaction into the only module that can reach a network. Both are impossible to forget; neither is defended in depth. Dropping the clinic predicate from `AccessScope.query` exposes every patient in both clinics and nothing catches it (D-085). We had been treating *unforgettable* as if it were *redundant*; the scenario asks about the second.

Privacy outside the front door (theme 3). Redaction-before-LLM is provable — no module but the wrapper may reach a model. The windows were the problem: crash logs carrying bound parameters (D-071), then the access log carrying a phone number we put in a URL ourselves (D-083). Each leak was in a sink our own code does not write to.

Numbers that mean something (theme 6). Risk has a deterministic floor a model cannot lower, and the floor is language-independent (D-072). Confidence is derived, not self-reported. Extraction and generation are separated in the code: highlights are character offsets into a pinned `source_version_number`, never model-paraphrased text, which is what makes scenario 16 answerable at all.

Feedback loops (theme 8). `NEVER_DAMPENED` floors a protected tag's learned weight, and we had been citing it as the answer to "what stops the ranking burying an allergy." It floors the wrong quantity: surfacing is a top-N cut, so other tags rising displaces an allergy with its own weight untouched, and one dismissal removed it from the card permanently. Protected classes now bypass ranking entirely (D-084) — learning orders the protected set, it no longer decides membership.

Identity and tenancy (theme 1). Scenario 5 asks config-versus-schema. *Schema: nothing* — every table carries `clinic_id` and the seed has run two clinics since Phase 1, so a third needs no migration. *Config: everything, which was the problem* — every value a clinic might differ on was a module constant, so "Clinic B keeps records for a year" was a deploy. `ClinicConfig` (D-086) makes volume and retention per-clinic, an absent row meaning defaults.

The more useful half was deciding what stays global. Redaction patterns, the protected classes from D-084, contradiction severities and the dosage reference are asserted *not* to be configurable, under one rule: **a clinic may change what it sees, never what it is protected from.** The gap keeping scenario 5 at PARTIAL is clinical vocabulary, still global — and the piece needing most care, because additions must be additive only or a clinic could remove `entity:allergy` and reach the safety floor sideways.

Where we are still weak, plainly. Degraded mode (theme 4) is handled for the model and absent for delivery — there is no sender, so scenario 11 cannot fully survive. Concurrency (theme 5) loses no updates but is not real-time. Scenario 7 remains DOES NOT: the scribe is post-hoc by construction, so an allergy at minute two is not in the Glance View until the consult ends. That is an architecture decision, stated rather than hedged, and `test_capture_timing.py` pins the boundary rather than papering over it.

9. Two failures the clinician sees, and the sinks we had not scanned

A pass over ease-of-use and leakage, asking only "what does a user experience when this goes wrong" rather than "does the logic hold".

A render crash white-screened the app. No error boundary existed, so any component throwing unmounted the whole tree. A clinician with a patient in the room got a blank page — and in a clinical record a blank page is indistinguishable from data loss, so the recovery they reach for is retyping a note they never lost. The boundary now leads with *nothing you saved has been lost*, which is true because writes commit server-side before the response returns, and it renders no error text at all: `error.message` can carry interpolated note content, and a stack trace on a shared consult-room laptop leaks in front of the patient.

A session timeout stranded the user. Sessions last 60 minutes with no refresh (D-016), which fires on a real clinic laptop most afternoons, and the 401 surfaced as "Token

expired" beside a chart that had silently stopped working. A security control that strands people is one people route around, so the sign-in screen now returns and states that saved work survived.

The sinks we had not scanned. Server logs were governed by `log_event`, crash output by D-071, URLs by D-083 — and the browser console by nothing. One `console.log(entry)` left in during debugging puts a full note somewhere most deployments forward to a third-party dashboard, and the app looks identical either way. A scan now fails the build on any console statement outside two content-free allowances.

What this pass confirmed rather than fixed. Patient-role isolation holds across nine endpoints probed with a patient token — zero fragments of staff notes, clinician sections or raw AI summaries in any response body — and the service worker is network-only for `/api`, so no patient data is written to disk on a shared device. Both are worth stating because "we checked and it held" is a different claim from "we assumed it held".

Known gap. React re-logs caught errors to the console in development builds, message included, and a boundary cannot suppress it. `Resilience.test.jsx` asserts that the leak happens, so it is visible in the suite rather than only in a document.

And the one that is ordinary rather than exceptional. This is a PWA for bedside use, so losing the network is not an edge case, and every such failure reached the clinician as the browser's own words — "Failed to fetch" — silent on the only question that matters mid-consult: did the note save. Reads and writes now say different things, because the reassurance differs. What is still missing is a queue, and the honest reason it was not attempted is that a durable outbox means unencrypted patient text in browser storage on a shared ward device — a data-protection decision, not an afternoon of work.

10. A second audit, and the pattern behind all six defects

Section 7 reported three defects found by probing rather than re-reading. We ran the exercise again against the twelve-capability list. Three more came out, and **all six share one cause.**

Contradiction detection could not see inside one entry (D-089). `detect()` compared entries pairwise and never an entry with itself — correct for typed notes, wrong the moment an entry is a transcript. `run_scribe` writes one Entry per consult, so an allergy at minute two against a prescription at minute nineteen was undetectable at any point in its life: not during the consult (nothing is incremental, which we had documented) and not after it, which we had not noticed. `test_capture_timing.py` passed throughout — it asserted the timing boundary and was blind to the detection gap beside it. Intra-entry comparison is now enabled, gated to three classes, with dose corrections requiring an explicit retraction cue so deliberation ("500 or 1000, depending on tolerance") stays quiet.

The abstention flag never fired for unspaced scripts (D-090). `is_unreadable` is the whole of our answer to "the transcript is trilingual" — when the tagger cannot read a turn it says so, rather than emitting an empty tag list indistinguishable from "nothing clinical was said". It measured substantiveness as `len(text.split()) >= 6`, so a Chinese or Japanese paragraph is one token, falls under the bar, and returns False. The comment beside our Malay vocabulary names Mandarin as uncovered and rests on this flag catching it. It did not. **A documented gap that a second mechanism silently fails to cover is worse than an undocumented one, because the documentation reads like a control.**

Exposure bias had a mitigation and no measurement (D-092). Now measured: displacement 0.15, exposure concentration **0.77**, blind-tag rate 0.16, zero protected classes displaced. The 0.77 is the bias as a number — ten of thirteen visible slots go to tags this clinic has already given feedback on. We are not claiming it is a good number; we have nothing to compare it to. We are claiming it moves when the system changes, which is what an argument cannot do. **We fell into the same trap building it**: the first evaluator ranked by score and took the top N, which is not what the Glance View does (D-084), and so reported that `entity:allergy` never reaches the card. False, and it would have gone in this brief as a finding.

One finding we did not fix, because the fix is bigger than the bug. Keyword tagging has no notion of negation — "no anaphylaxis" emits `symptom:anaphylaxis` — a known limitation with a stated reason. What was *not* documented is its interaction with the protection list: `symptom:anaphylaxis` is in `NEVER_DAMPENED`, so a note ruling anaphylaxis out produces a tag that can never be learned down. The clinician dismisses it and the floor discards the dismissal — **the anti-alert-fatigue mechanism manufacturing a permanent false positive.**

The pattern, stated once. All six were invisible to their own tests for the same reason: **the test used the shape of the case its author had in mind.** Cross-entry contradiction tests used two entries; abstention tests used romanised Latin script; the evaluator modelled the card its author assumed. None were careless — each was written from inside the assumption it needed to escape, which more of the same tests would not have fixed.

So we changed their shape rather than their number, and it found two more. *A phone number could hide behind an en-dash (D-095):* every separator class in `redaction.py` is spelled in ASCII, and while `\s` is Unicode-aware the dash class is not, so `hp 9123–4567` passed untouched and `find_residual_phi` reported clean. The second half is the real finding — that function is both the fail-closed tripwire and the oracle our property tests assert against, and it shares its regexes with the redactor. **A check and its own test must not share an implementation**, or the gap is invisible twice. *Clinic isolation is now enumerated, not sampled (D-096):* against the exact mutation the feedback describes, hand-written RBAC tests raise 15 failures and the enumerated matrix raises 48 — both catch it, only one tells you what it exposed.

Four more properties held, and that is also a result (D-097): content round-trip through the real API, analyser totality and determinism over the full unicode range, revision-history invariants over generated edit/revert sequences, and log hygiene greping every logger at DEBUG for synthetic identifiers. Nothing leaked. **Every property was mutation-checked**, because a test that has never failed is a test whose teeth are unmeasured.

We also found a leak we chose **not** to fix: space-separated identifiers (`900101 01 5432`, which is what a patient reading an IC aloud transcribes to) survive redaction. Widening the pattern puts every run of grouped clinical digits at risk of becoming `[ID_1]` — a narrow privacy gap traded for a broad accuracy one, which is exactly what the hint warned against.

11. A final pass, and the four defects a green suite was hiding

We ran the exercise one more time before submitting. Not by re-reading the code — we had done that twice — but by running the system on clinical text nobody had run it on: a three-drug reconciliation list, a patient editing her own note, a 400-day-old entry going through decay. Four defects came out, and **all four were live while 847 backend and 67 frontend tests passed.** None would have been found by making the existing tests stricter.

Section 7 ended by saying we expected a fourth overclaim to exist and named the rows we would probe first: **12 and 15**. Two of these four are in scenario 12. We record that not as a good prediction but as the opposite — we wrote down where we thought the build was weakest and then shipped two more rounds of work before going to look.

The patient was warned that her own note had been corrected (D-100). Two constants named `PATIENT_FACING_TYPES` existed, in `core/enums` and in `security/policy`, holding different sets — one meaning *readable by* the patient, the other *written for* her. Both were right for their own module. `delivery.py` imported the first, so a note the patient typed herself was treated as clinic-authored content that had failed to reach her. Her clinician's card listed it under "not yet opened by the patient". Worse, when she edited it, her own view led with *"This was updated after you last read it. If you were following the earlier version, stop and read this one."* That banner is the highest-severity thing this build says to a patient; it means *the clinic changed something you already acted on, possibly a dose*. It was firing because she added a line to her own note — an alert aimed at the one reader in the system with no provenance rail to check it against. We deleted the duplicate constant rather than renaming it, because a third name keeps the hazard.

A dose belonged to whichever drug came first (D-101). Two modules extract medication-plus-dose. They shared a regex — with a comment saying so — and disagreed about which drug a dose belongs to. `contradictions` gave the first dose in a sentence to every drug in it, so *"Continue metformin 1g BD, amlodipine 5mg OD, atorvastatin 20mg ON"* generated the claim `amlodipine 1g` and reported a **HIGH-severity disagreement between two entries that agree**, citing a dose that does not exist for that drug. Our own docstring claims that module's "failure mode is silence, never a wrong answer". Untrue, and it is the exact false positive we argue elsewhere is worse than a gap. The dosage checker had the mirror fault: its window ran past the next drug name, so *"metformin and amlodipine 5mg"* read as metformin 5mg and **blocked a patient-facing write on correct prose**. One shared extractor now binds each dose to its nearest drug, bounded on both sides — which also closed a silent miss, since dose-before-drug (*"take 20mg atorvastatin"*, how instructions are actually written) carried no dose at all, leaving a decimal slip in that phrasing invisible to the gate built to catch decimal slips.

Decay walked around our provenance mechanism (D-102). Scenario 16 asks what happens when a highlight cites a source that has since changed. We answered it for edits, tested it, and scored ourselves SURVIVES. `decay.compress` replaces an entry's content with an extractive summary and — correctly — does not create a version, because archival is not an authorship event. But staleness was `source_version_number != version_number`, and compression moves neither. So a cold entry reported `stale: false` while every character of its content had been replaced: a highlight anchored to *"mild ankle swelling"* resolved to `'ing in the evenings'` at the same offsets, and clicking it drew a box around that fragment in the timeline. That is precisely the "silently point at different text" outcome the mechanism exists to prevent. Fixing it exposed that the UI half was wrong for **edits** too — the card showed the side-by-side while the timeline underneath highlighted the old offsets in the new text.

The pattern, which is the actual finding. Three of the four live in a seam between two modules that are each correct alone: two constants sharing a name, two extractors sharing a regex but not a rule, and a lifecycle transition that bypasses a mechanism watching a different variable. Section 10 said all six defects of the previous pass shared one cause. This is a different one, and it is the one we would now expect to keep producing bugs: **our per-module reasoning is dense and mostly right, and our joins are unexamined.** Every module in this repo carries a long docstring defending its own behaviour. None of them describes what it assumes about the module next to it.

An assumption that no longer stands. We wrote, more than once, that this build's failure mode is silence — that a gap in a watchlist or a vocabulary makes the system less helpful, never wrong. That is true of the vocabulary and it was not true of the code around it. Two of these four produced confident false statements to a clinician, and one of those blocked their work.

What this pass did not cover, stated so the absence of a finding is not read as a clean bill: the voice-capture and ASR pipeline, the concurrency paths beyond their existing tests, and the learning loop past its accumulator were read but not probed with the same adversarial input that produced the four above. On the evidence — three of four in module seams — that is where we would look next, and specifically at what `capture` assumes about `scribe`.